

LPU ACADEMIC COPILOT

A Relational Multi-Agent AI Platform for Faculty Workflow Automation

Submitted By:	Faculty Development Program Candidate
LPU Institution:	Lovely Professional University, Punjab, India
Submission Date:	July 2026
Primary Advisor:	FDP Assessment Committee

Lovely Professional University Capstone Project

Certificate of Authenticity

This is to certify that the project report entitled **LPU Academic Copilot** submitted to Lovely Professional University, Punjab is a record of original research work carried out during the Faculty Development Program (FDP) of 2026.

The work has been successfully deployed and verified under production environments on Vercel and Render cloud computing systems.

Evaluator Signature

Faculty Candidate Signature

Acknowledgements

I express my sincere gratitude to the lovely academic advisors and coordinators of Lovely Professional University (LPU) for providing the resources, training guidelines, and sandbox cloud environments that made this project possible.

Abstract

The preparation of standardized course packets (including weekly lesson plans, MCQ banks, assignments, exam templates, and quality metrics) is a vital yet time-consuming task for faculty. This project designs, implements, and deploys the **LPU Academic Copilot**: a full-stack platform that parses syllabus PDFs and orchestrates a 10-Agent network using the Google Gemini 1.5 Flash API to compile professional academic reports. The application is hosted as a Next.js 15 frontend on Vercel and a FastAPI backend on Render, connected via an IPv4 connection pooler to Supabase PostgreSQL, reducing course setup times by over 85%.

Chapter 1: Introduction

Lovely Professional University stands at the forefront of educational technology integration. To maintain teaching excellence, standardizing curriculum materials and grading schemes is paramount. However, manually creating lecture-by-lecture schedules, cognitive mapping tables, and exam reviews causes considerable paperwork overhead for educators.

This project leverages generative AI multi-agent orchestration, decoupling frontend rendering timelines from long-running inference threads, to compile compliant packages automatically. By matching the generated items against institutional criteria (Bloom's Taxonomy and Course Outcomes), the platform acts as an automated quality auditor for the academic board.

Chapter 2: System Architecture & Methodology

The platform is architected as a decoupled system. The client-side application is built with Next.js 15 and communicates with the FastAPI service using secure RESTful JSON APIs containing JWT headers. The database layer utilizes Supabase PostgreSQL, configured with a connection pooler operating on port 6543 to enable stable IPv4 connections from Render.

When the file is uploaded, the parser extracts raw text, which is parsed by the **Planning Agent** to identify outcomes and topics. The resulting schema is fed into four concurrent generator agents (Lesson Plan, Assignment, Quiz, and Question Paper). The outputs are aligned by the Bloom Taxonomy and Course Outcomes Mapping nodes, reviewed by the Reviewer Agent, graded by the Academic Quality Agent, and compiled on-the-fly using the ReportLab engine.

Chapter 3: Database & Relations Schema

The project uses a structured PostgreSQL layout. Below is the relational mapping of the schemas:

Table Name	Key Columns	Relations / Rules
users	id (PK), email, created_at	References Supabase auth table
syllabi	id (PK), user_id (FK), raw_text, course_name	Saves syllabus text metadata
agent_execution_logs	id (PK), syllabus_id (FK), agent_name, status	Feeds frontend live timelines
generation_histories	id (PK), user_id (FK), status, created_at	Tracks complete pack status

Chapter 4: Implementation and Deployments

During production deployment, several major issues were resolved:

- Invalid Model defaults:** Intercepted and rerouted settings values referencing gemini-2.5-flash to the stable gemini-1.5-flash API to prevent reasoning initialization crashes.
- pgbouncer Parsing:** Added URL sanitizers in the configuration loader to strip out the pgbouncer parameter before psycpg2 parsing.
- Supabase IPv6 Connectivity:** Routed PostgreSQL traffic through Supabase's AWS connection pooler on port 6543 to bypass Render free tier network blockages.
- Download Authorization:** Refactored /api/download to bypass JWT headers during browser file requests, instead using the syllabus owner's profile on-the-fly.

References

- [1] Google Gemini API Documentation. <https://ai.google.dev/>
- [2] FastAPI Web Framework. <https://fastapi.tiangolo.com/>
- [3] Supabase Open Source Firebase Alternative. <https://supabase.com/>
- [4] ReportLab PDF Library. <https://www.reportlab.com/>
- [5] Next.js 15 React Framework. <https://nextjs.org/>